МИНОБРНАУКИ РОССИИ

федеральное государственное бюджетное образовательное учреждение высшего образования «Московский государственный технологический университет «СТАНКИН»

(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных технологий

Кафедра
Прикладная информатика

Основная образовательная программа 09.03.03
«Прикладная информатика»

Отчет по дисциплине «Информационные системы и технологии»

Проектная работа: «Информационная система оформления ОСАГО»

Студент
группы ИДБ-23-13

Таган А.М
Киселев Д.А.
Романов Н.И.
Усков М.В.
Мехова А.С.

Преподаватель

Бычков С. Ю.

Москва, 2026

Ревью диаграмм компонентов (C4, уровень 3)

ИС ОСАГО, ветки new era — итоги обсуждений

Автор: тимлид команды. Участники: Дима (API), Миша (Worker), Амелия (Backoffice), Никита (Web App).

Как проходило ревью

Каждую диаграмму компонентов сверяли с утвержденными диаграммами контекста и контейнеров (context.puml, container.puml). Диаграмма компонентов раскрывает ровно один контейнер, поэтому проверяли три вещи: (1) все связи этого контейнера с уровня 2 перенесены один в один — те же внешние элементы, направления, подписи и протоколы, а асинхронные связи (пунктир на уровне 2) остались пунктирными; (2) каждая такая связь выходит из конкретного компонента или входит в него; (3) внутри контейнера соблюдены слои и нет противоречий с кодом в ветках. Ниже по каждому — исходная диаграмма, исправленная версия и короткие пояснения. Это сводка нескольких обсуждений.

Обозначения на исправленных диаграммах: оранжевая рамка — компонент изменен или добавлен на ревью; черная сплошная линия — связь, перенесенная с диаграммы контейнеров; черный пунктир — асинхронная связь уровня 2; серая линия — внутренняя связь; фиолетовый пунктир — проверка прав/роли; серый пунктир — запись в аудит. Подписи внешних связей дословно совпадают с container.puml. К каждой диаграмме приложены файлы .drawio и .puml; в отчете показаны обе версии — нарисованная в draw.io и построенная PlantUML из .puml.

Общие решения для всей команды

- **Внешние связи** — один в один с уровнем 2 — сколько связей у контейнера на диаграмме контейнеров, столько и на его диаграмме компонентов, с теми же подписями и протоколами. Ничего нового за пределами контейнера не придумываем. Если к одной базе обращаются несколько репозиториев, они сходятся в одну линию — это одна связь уровня 2.
- **Пользователей не рисуем** — на диаграммах компонентов нет акторов: они уже показаны на уровнях 1–2. Поэтому связи «человек → контейнер» (в том числе «Страховой агент → API», КВ) на компонентных не показаны; роли перечислены в описании AppRouter и AccessService.
- **Имена** — как на диаграмме контейнеров: Web App, Backoffice, API, Worker, DB_USERS, DB_TARIFS, DB_AUDIT, DB_FILES, АИС ОСАГО (РСА), Платежная система, SMS-шлюз, E-mail система. Варианты BD_* и «BD_DTP» убраны.

- **Очередь задач** — outbox в DB_AUDIT, без Redis. Со стороны API это связь «API → Worker: ставит задачи» (из TaskQueueFacade), со стороны Worker — «Worker → DB_AUDIT: забирает задачи из outbox» (через OutboxPoller и AuditRepository).
- **Права** — в API один общий AccessService (RBAC) читает роли через UserRepository; на фронтах страницы закрывает AppRouter по роли, токен подставляет HttpClient.
- **Аудит и данные** — в API аудит и очередь пишет AccidentRepository (DB_AUDIT, как в исходной диаграмме Димы), тарифы — TariffRepository, пользователи и полисы — UserRepository, файлы — FileRepository; у Worker аудит и outbox — AuditRepository, транзакции и платежи — TransactionRepository.
- **Слои** — Controller → Service → Facade/Repository → БД; внешние системы только через фасады; сервис не ходит в БД напрямую.

1. Дима — API

Дима раскрывал контейнер API, но показал только контур ДТП, поэтому большая часть связей API с уровня 2 (SMS-шлюз, платежная система и webhook, E-mail, DB_USERS, DB_TARIFS, Backoffice) не была отражена. На обсуждениях договорились не раздувать диаграмму: его компоненты не трогаем, добавляем только то, что нужно, чтобы связи уровня 2 выходили из конкретных компонентов, и общую проверку прав. Контуры финансов, полисов, агентов и отдельные стратегии пока не рисуем — они уйдут на второй этап.

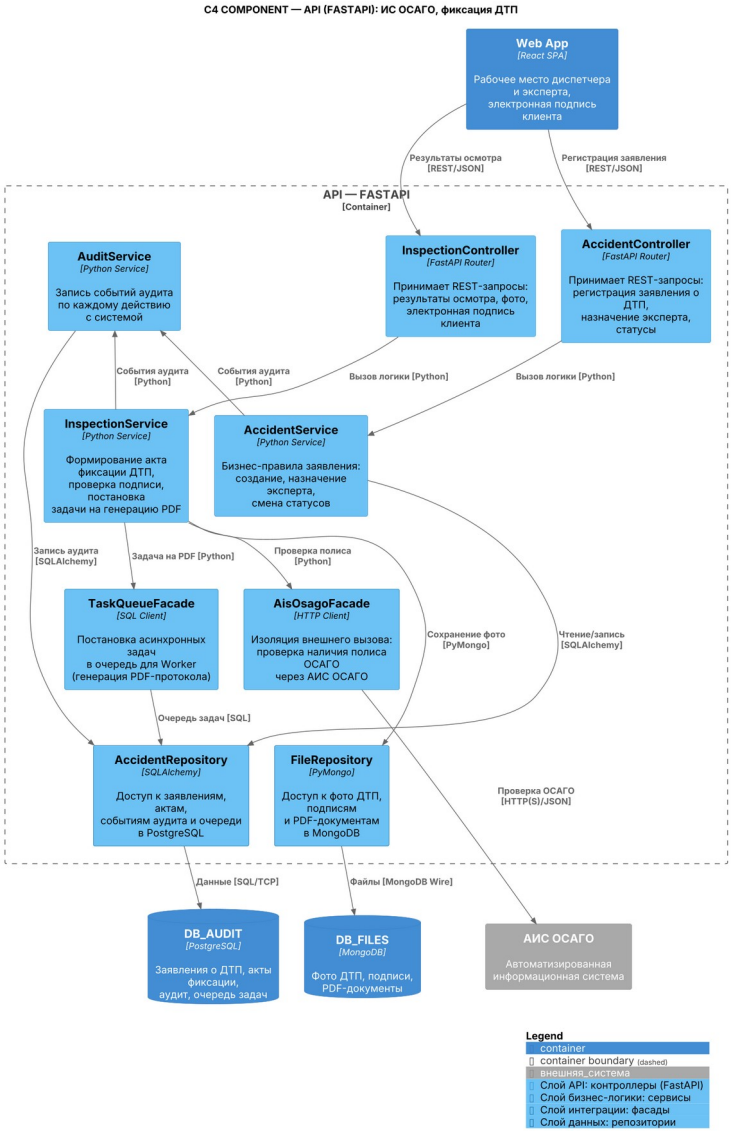


Рисунок 1 — Исходная диаграмма Димы

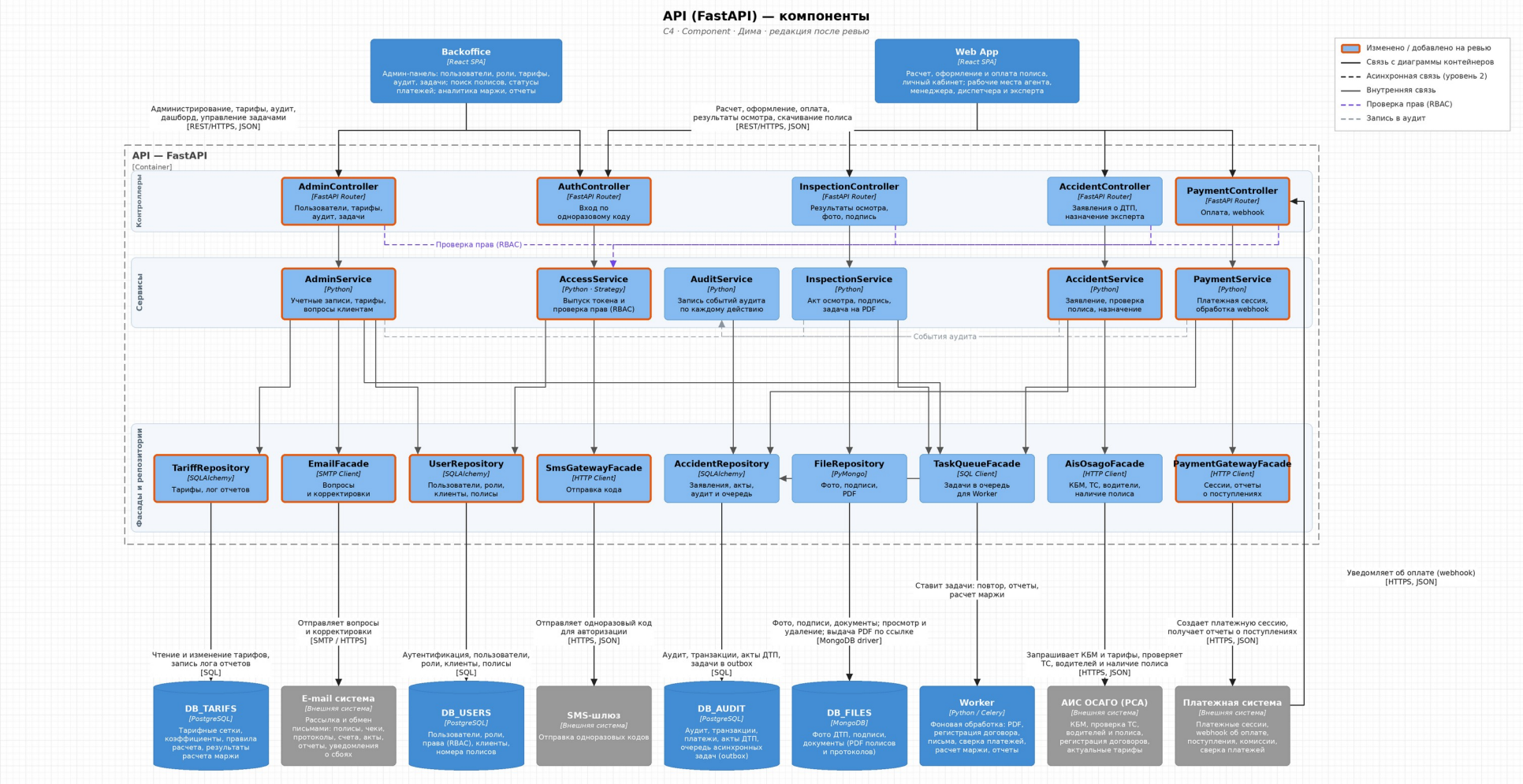


Рисунок 2 — Диаграмма компонентов API после ревью

Та же диаграмма в PlantUML (файл .puml в папке puml). Содержимое и подписи связей те же, но расположение блоков и линий выбирает сам PlantUML, поэтому вид отличается от версии draw.io.

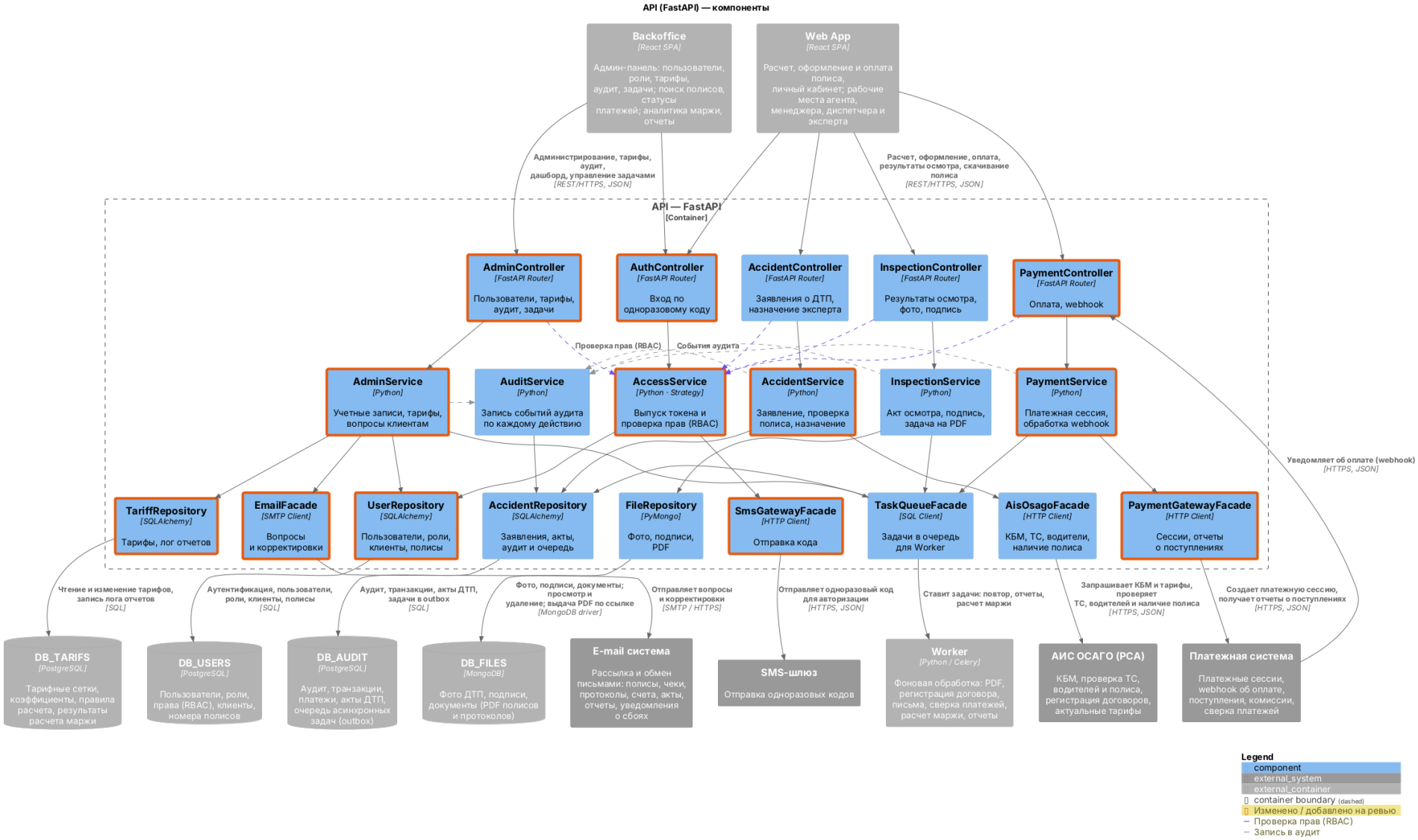


Рисунок 3 — Диаграмма компонентов API после ревью — вид в PlantUML

Описание компонентов

Исходные компоненты Димы (контур ДТП) остались как были; ниже — только то, что добавлено, чтобы закрыть связи уровня 2.

- **Controllers** — AccidentController, InspectionController (как у Димы); AuthController — вход по одноразовому коду; AdminController — запросы Backoffice (пользователи, тарифы, аудит, задачи); PaymentController — оплата и webhook платежной системы.
- **Services** — AccidentService (заявление, проверка полиса, назначение эксперта), InspectionService (акт, подпись, задача на PDF), AuditService — как у Димы; AccessService — выпуск токена и RBAC; AdminService — учетные записи, тарифы, вопросы клиентам; PaymentService — платежная сессия и обработка webhook.
- **Facades и repositories** — AisOsagoFacade, TaskQueueFacade, AccidentRepository, FileRepository — как у Димы; SmsGatewayFacade, PaymentGatewayFacade, EmailFacade — изоляция вызовов внешних систем; UserRepository и TariffRepository — доступ к DB_USERS и DB_TARIFS.

Сводка изменений

№	Что было	Что стало и почему
1	Показан только контур ДТП; нет связей API с Backoffice, SMS-шлюзом, платежной системой (включая webhook), E-mail, DB_USERS и DB_TARIFS	Добавлены три небольших цепочки: вход по коду (AuthController → AccessService → SmsGatewayFacade, UserRepository), администрирование для Backoffice (AdminController → AdminService → TariffRepository, UserRepository, EmailFacade) и оплата (PaymentController → PaymentService → PaymentGatewayFacade, плюс постановка задач после оплаты). Все 12 связей API с уровня 2 выходят из конкретных компонентов с теми же подписями.
2	Полис в АИС ОСАГО проверяет InspectionService на этапе осмотра	Проверка перенесена в AccidentService при регистрации заявления: без действующего полиса заявление дальше не идет, эксперт не выезжает зря.
3	Нет проверки прав	Добавлен AccessService (RBAC): контроллеры проверяют права через него до бизнес-логики. Читает роли через UserRepository из DB_USERS, как на уровне 2.
4	Имена «АИС ОСАГО», подписи связей в своей формулировке	Имена и подписи внешних связей приведены к container.puml дословно, включая протоколы (SQL, MongoDB driver, HTTPS JSON, SMTP / HTTPS).

Что поправить в коде

- inspection_service.submit_results: убрать вызов self._ais.check_policy(claim) и перенести его в accident_service.register_claim.
- Добавить проверку прав через AccessService в роутеры accident/inspection.

2. Миша — Worker

Миша раскрывал контейнер Worker. Исходная диаграмма описывала только расчет маржи и сверку платежей, а очередь была нарисована как отдельный контейнер на Redis. По уровню 2 у Worker семь связей — все они теперь на диаграмме.

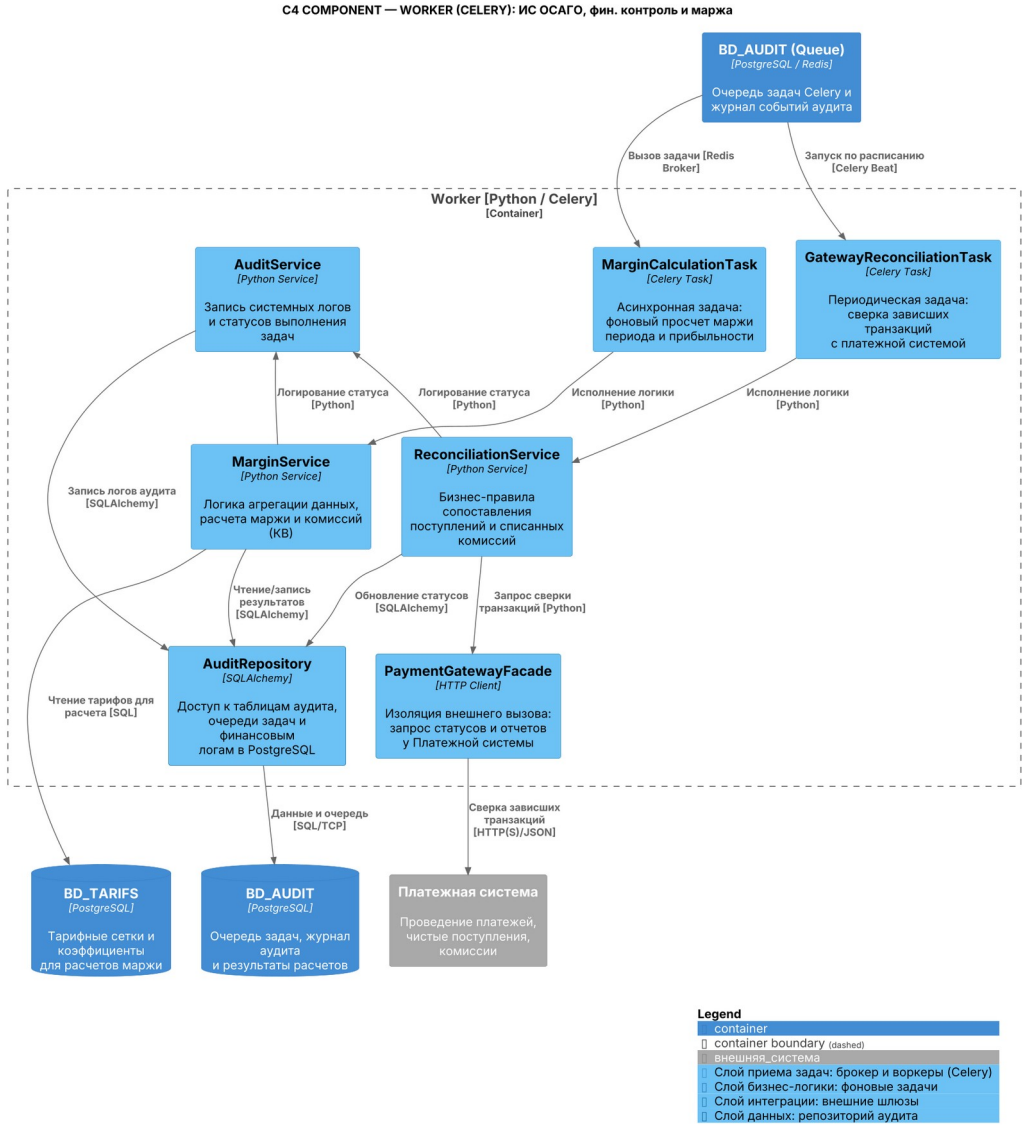


Рисунок 4 — Исходная диаграмма Миши — Worker

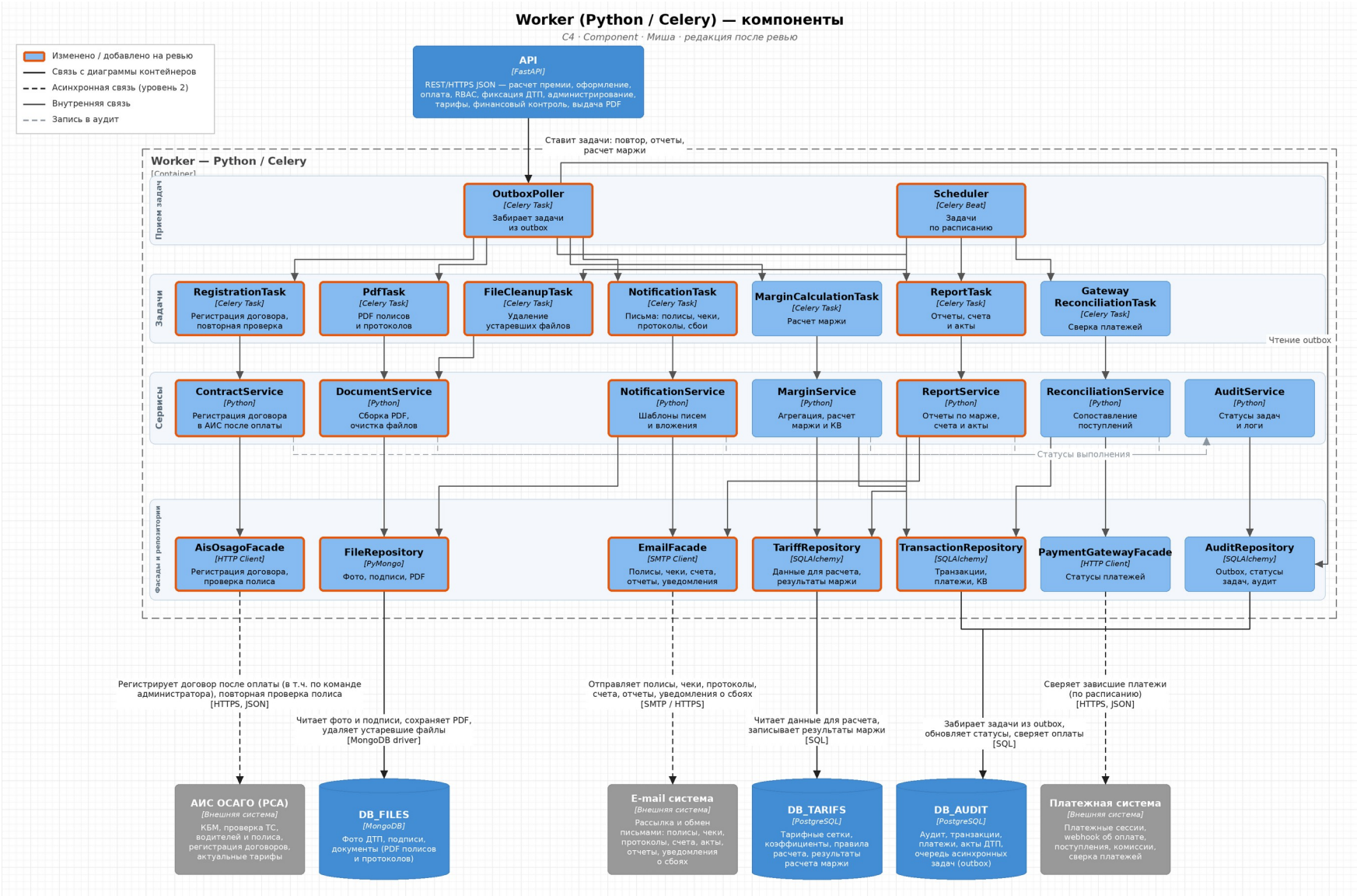


Рисунок 5 — Диаграмма компонентов Worker после ревью

Та же диаграмма в PlantUML (файл .puml в папке puml). Содержимое и подписи связей те же, но расположение блоков и линий выбирает сам PlantUML, поэтому вид отличается от версии draw.io.

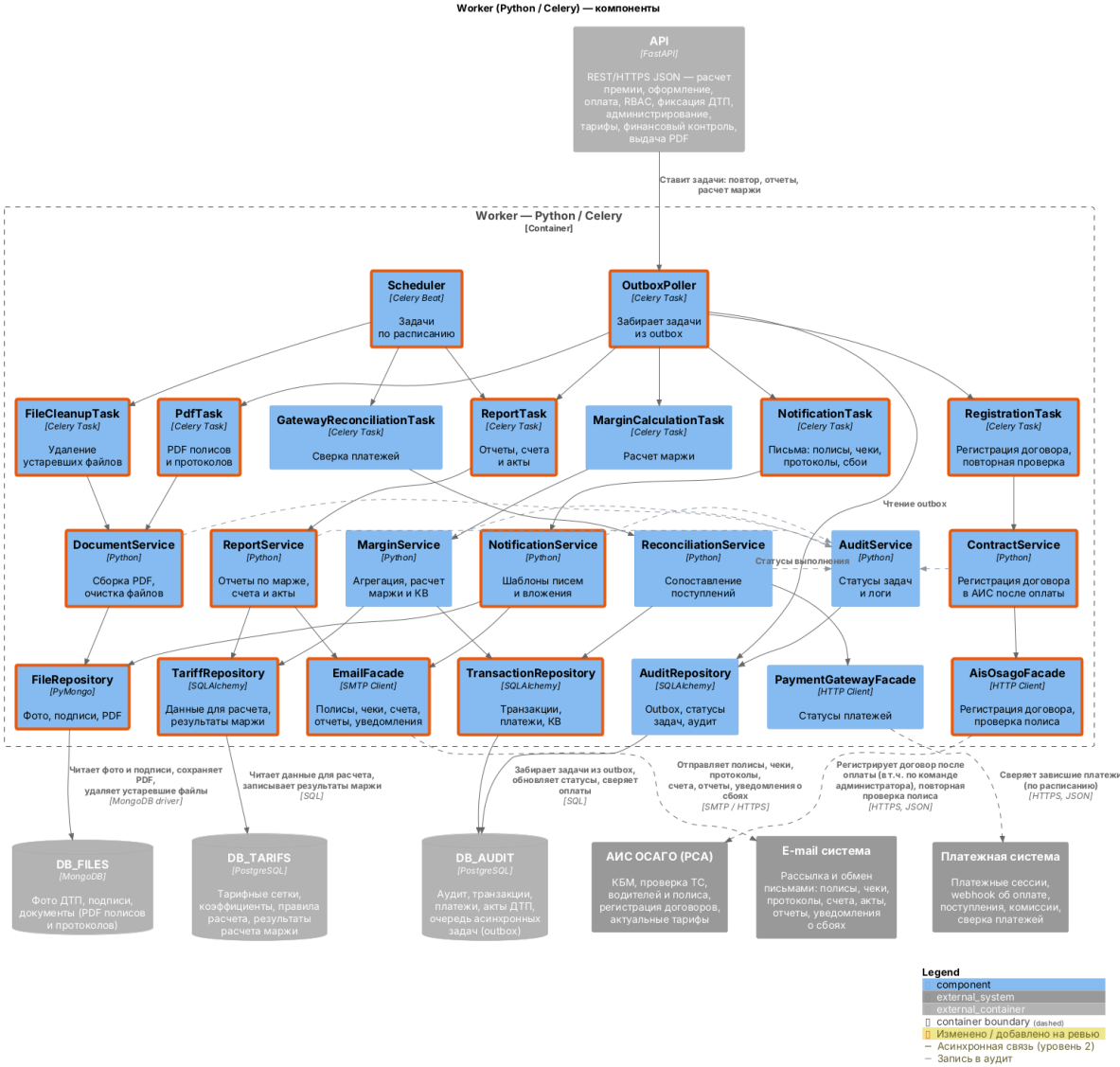


Рисунок 6 — Диаграмма компонентов Worker после ревью — вид в PlantUML

Описание компонентов

Worker — фоновый обработчик: принимает задачи, исполняет их и ходит в базы и внешние системы.

- **Прием задач** — OutboxPoller забирает задачи из outbox в DB_AUDIT (поставил API); Scheduler (Celery Beat) запускает периодические задачи.
- **Задачи** — RegistrationTask (регистрация договора, повторная проверка), PdfTask, FileCleanupTask, NotificationTask (письма), MarginCalculationTask, ReportTask, GatewayReconciliationTask (сверка платежей).
- **Сервисы** — ContractService, DocumentService, NotificationService, MarginService, ReportService, ReconciliationService исполняют бизнес-логику задач; AuditService пишет статусы и логи.
- **Фасады и репозитории** — AisOsagoFacade, PaymentGatewayFacade, EmailFacade (внешние системы, асинхронно), FileRepository (DB_FILES), TariffRepository (DB_TARIFS), TransactionRepository и AuditRepository (DB_AUDIT).

Сводка изменений

№	Что было	Что стало и почему
1	Внешний контейнер «BD_AUDIT (Queue), PostgreSQL / Redis», брокер Redis, плюс вторая база BD_AUDIT	Redis на уровне 2 нет, база одна — DB_AUDIT. Вход в Worker — связь уровня 2 «API → Worker: ставит задачи…» в OutboxPoller; задачи читаются из outbox через AuditRepository, наружу — связь «Worker → DB_AUDIT: забирает задачи из outbox, обновляет статусы, сверяет оплаты».
2	Worker делает только маржу и сверку	По уровню 2 Worker также формирует PDF, регистрирует договор в АИС (в т.ч. повторная проверка полиса), отправляет письма и отчеты и удаляет устаревшие файлы. Добавлены RegistrationTask, PdfTask, FileCleanupTask, NotificationTask, ReportTask и сервисы к ним; связи с АИС ОСАГО, E-mail системой и DB_FILES перенесены с уровня 2.
3	Периодический запуск сверки идет «из очереди» через Celery Beat	Добавлен Scheduler (Celery Beat) внутри Worker: сверка, отчеты за период, очистка файлов.
4	MarginService ходит в BD_TARIFS напрямую по SQL	Через TariffRepository; результаты маржи пишутся в DB_TARIFS, как на уровне 2 («записывает результаты маржи»).
5	Аудит, транзакции и очередь в одном AuditRepository	Транзакции, платежи и КВ — TransactionRepository; AuditRepository — outbox, статусы задач, аудит. Оба идут в DB_AUDIT одной связью.
6	Внешние вызовы подписаны своими словами	Подписи и протоколы — дословно с уровня 2; связи с АИС ОСАГО, платежной системой и E-mail остались асинхронными (пунктир).

3. Амелия — Backoffice

Зона Амелии — контейнер Backoffice, но в исходной диаграмме раскрыт API: контроллеры, сервисы и репозитории админ-контура, а админ-панель показана снаружи как Web App. Диаграмму переписали как компоненты Backoffice; содержимое исходника не потеряно — оно вошло в общую диаграмму API.

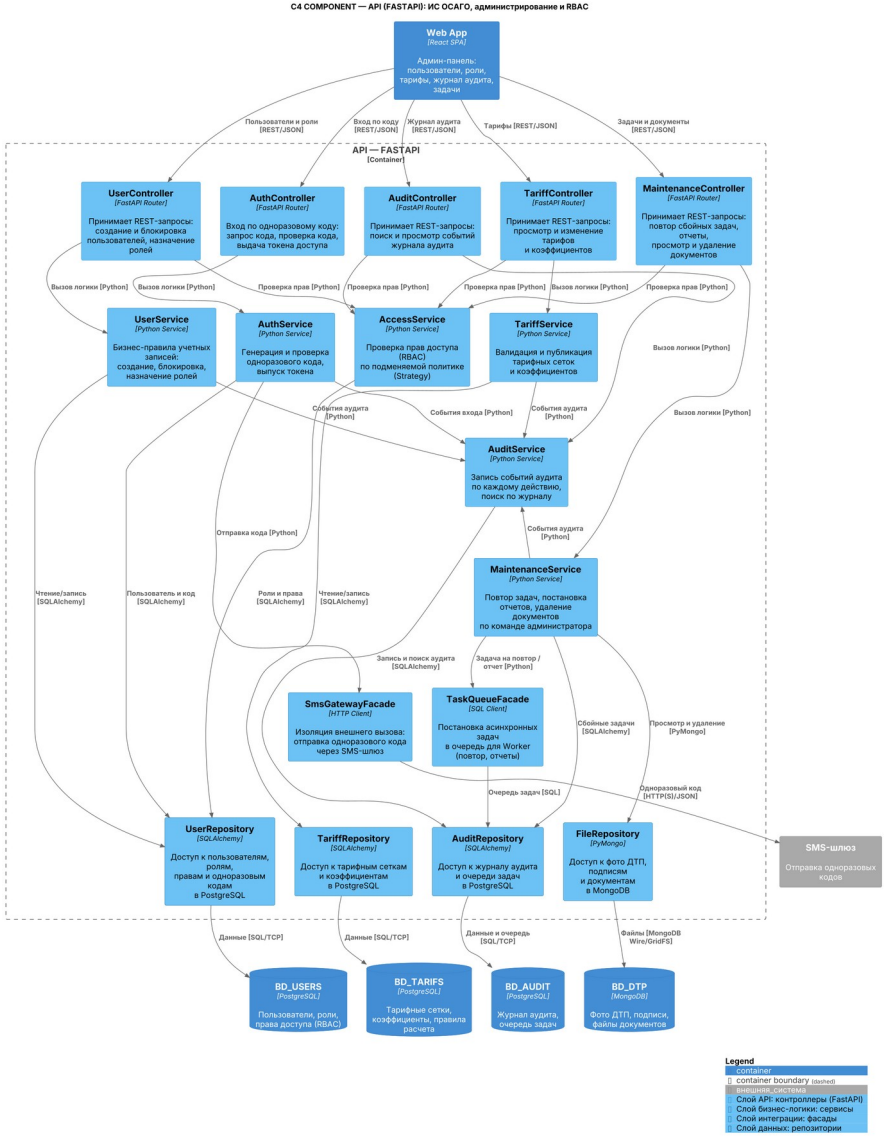


Рисунок 7 — Исходная диаграмма Амелии

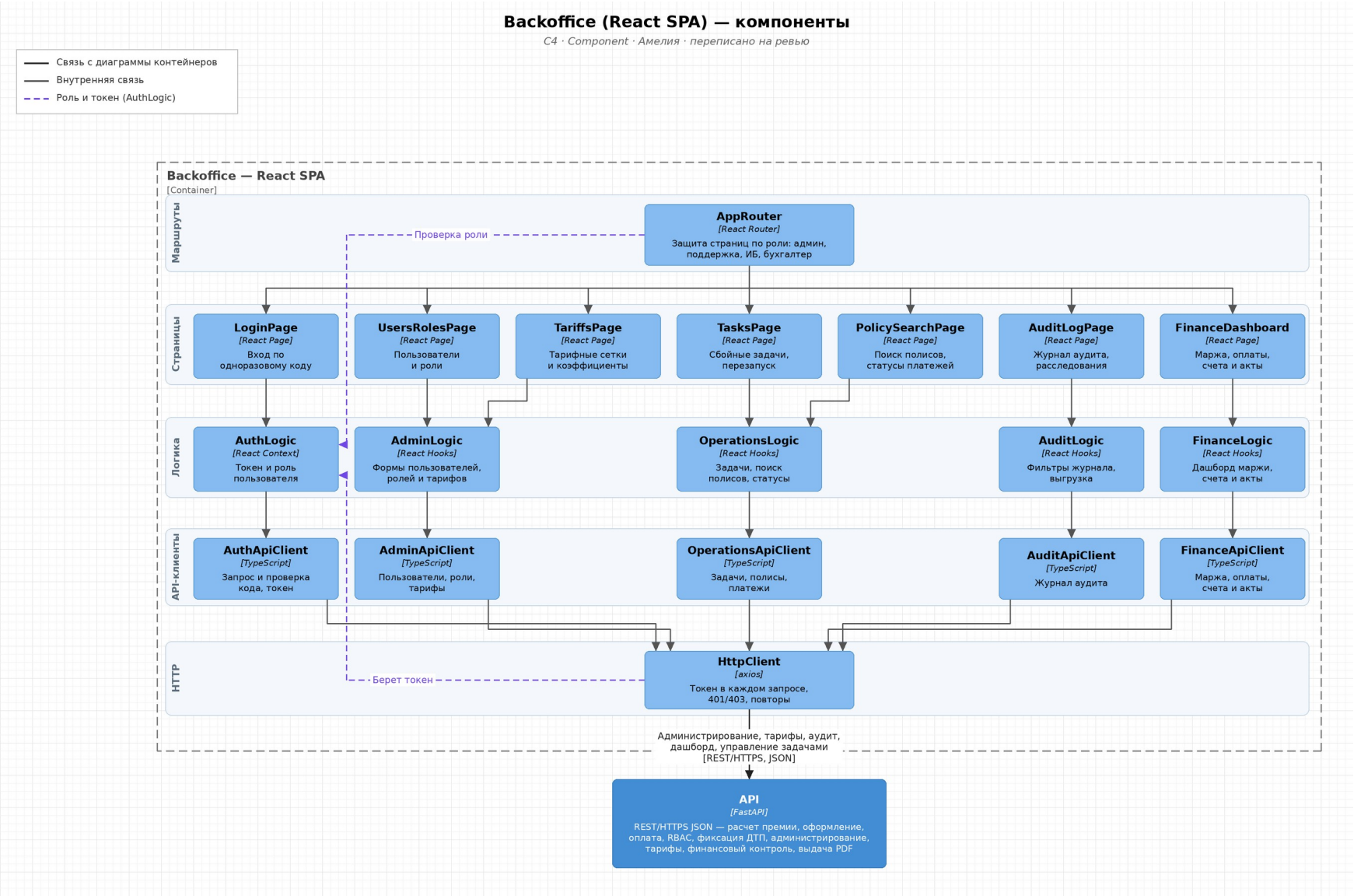


Рисунок 8 — Диаграмма компонентов Backoffice после ревью

Та же диаграмма в PlantUML (файл .puml в папке puml). Содержимое и подписи связей те же, но расположение блоков и линий выбирает сам PlantUML, поэтому вид отличается от версии draw.io.

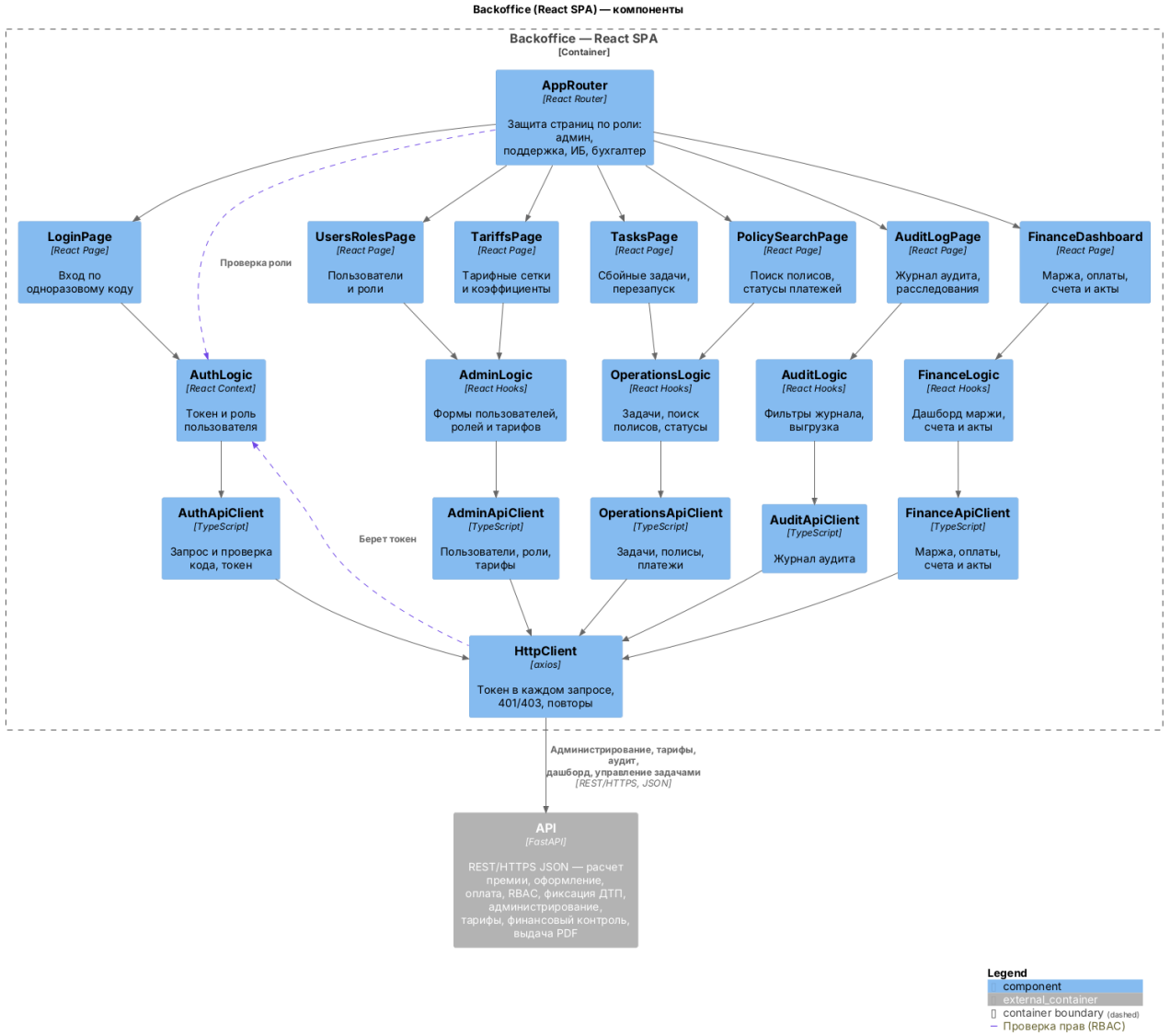


Рисунок 9 — Диаграмма компонентов Backoffice после ревью — вид в PlantUML

Описание компонентов

Backoffice — React SPA для сотрудников: администратора, поддержки, ИБ и бухгалтера. Единственная внешняя связь — вызовы API.

- AppRouter пускает на страницы по роли; страницы — LoginPage, UsersRolesPage, TariffsPage, TasksPage, PolicySearchPage, AuditLogPage, FinanceDashboard.
- **Логика** — AuthLogic (токен и роль), AdminLogic, OperationsLogic, AuditLogic, FinanceLogic; API-клиенты — по одному на область, все ходят через HttpClient (axios), который подставляет токен и обрабатывает 401/403.

Сводка изменений

№	Что было	Что стало и почему
1	Внутри границы — компоненты API (AuthController, UserController, TariffController, AuditController, MaintenanceController, сервисы, репозитории)	Внутри границы теперь Backoffice: AppRouter, страницы, логика, API-клиенты и HttpClient. Компоненты API из исходника вошли в диаграмму API: AuthController/AuthService — как были; UserController, TariffController, AuditController и MaintenanceController объединены в AdminController, а UserService, TariffService и MaintenanceService — в AdminService; AccessService, AuditService и репозитории сохранены.
2	Админ-панель обозначена как внешний Web App	Web App и Backoffice — разные контейнеры. У Backoffice на уровне 2 одна связь с системой — «Backoffice → API: администрирование, тарифы, аудит, дашборд, управление задачами [REST/HTTPS, JSON]», она и показана.
3	Роли Backoffice не видны	Страницы разложены по ролям уровня 2: администратор — UsersRolesPage, TariffsPage, TasksPage; оператор поддержки — PolicySearchPage, TasksPage; сотрудник ИБ — AuditLogPage; бухгалтер — FinanceDashboard. AppRouter пускает на страницы по роли.
4	Вход по одноразовому коду и RBAC были сильной стороной исходника	Идея сохранена: на стороне Backoffice добавлены LoginPage, AuthLogic, AuthApiClient и HttpClient с токеном; сама проверка прав осталась в API (AccessService).

4. Никита — Web App

Никита раскрывал Web App, но только контур ДТП. По уровню 2 Web App — это еще расчет, оформление и оплата полиса, личный кабинет и рабочие места агента и менеджера; связь Web App с API подписана именно так. Контур ДТП из исходника сохранен.

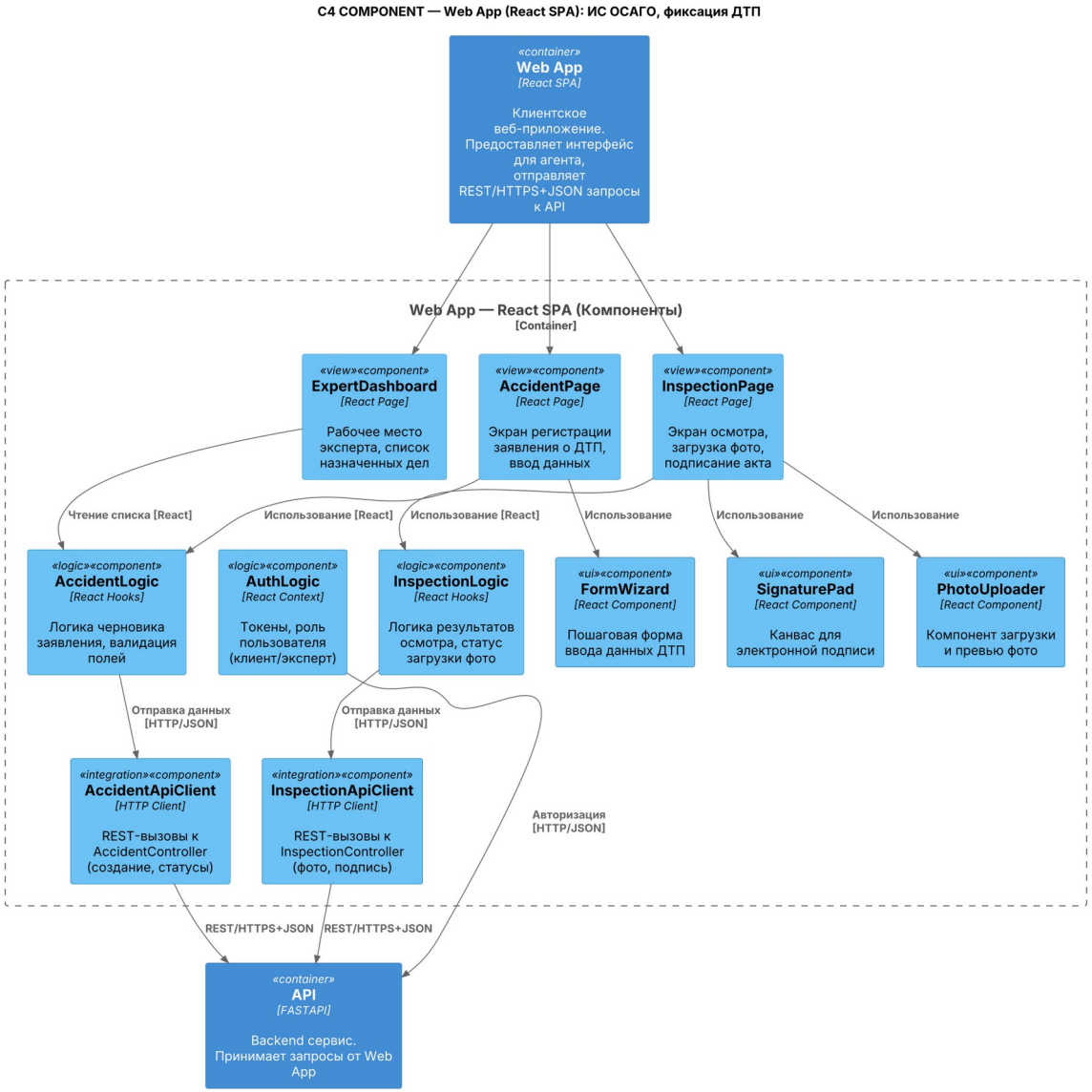


Рисунок 10 — Исходная диаграмма Никиты

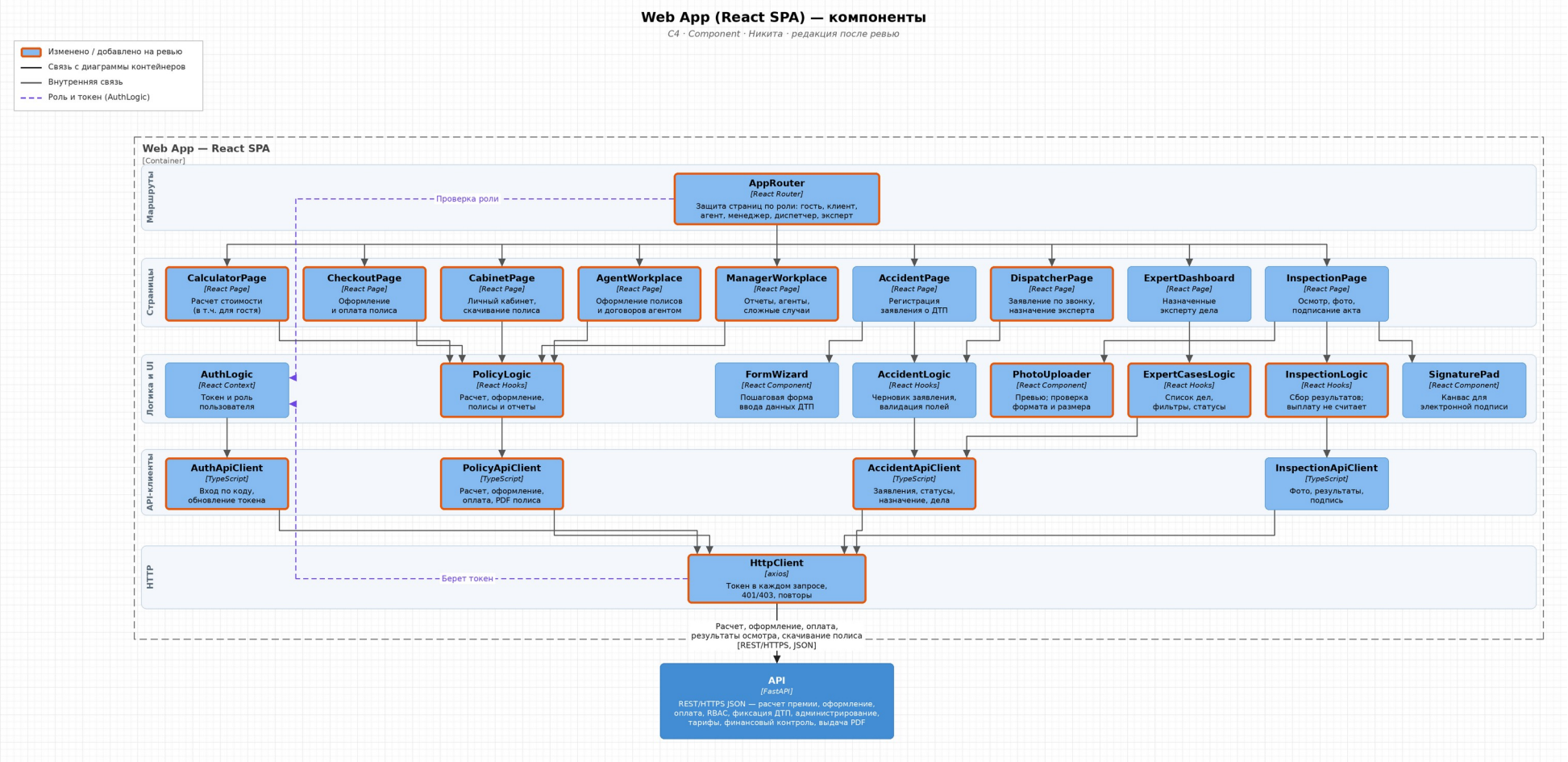


Рисунок 11 — Диаграмма компонентов Web App после ревью

Та же диаграмма в PlantUML (файл .puml в папке puml). Содержимое и подписи связей те же, но расположение блоков и линий выбирает сам PlantUML, поэтому вид отличается от версии draw.io.

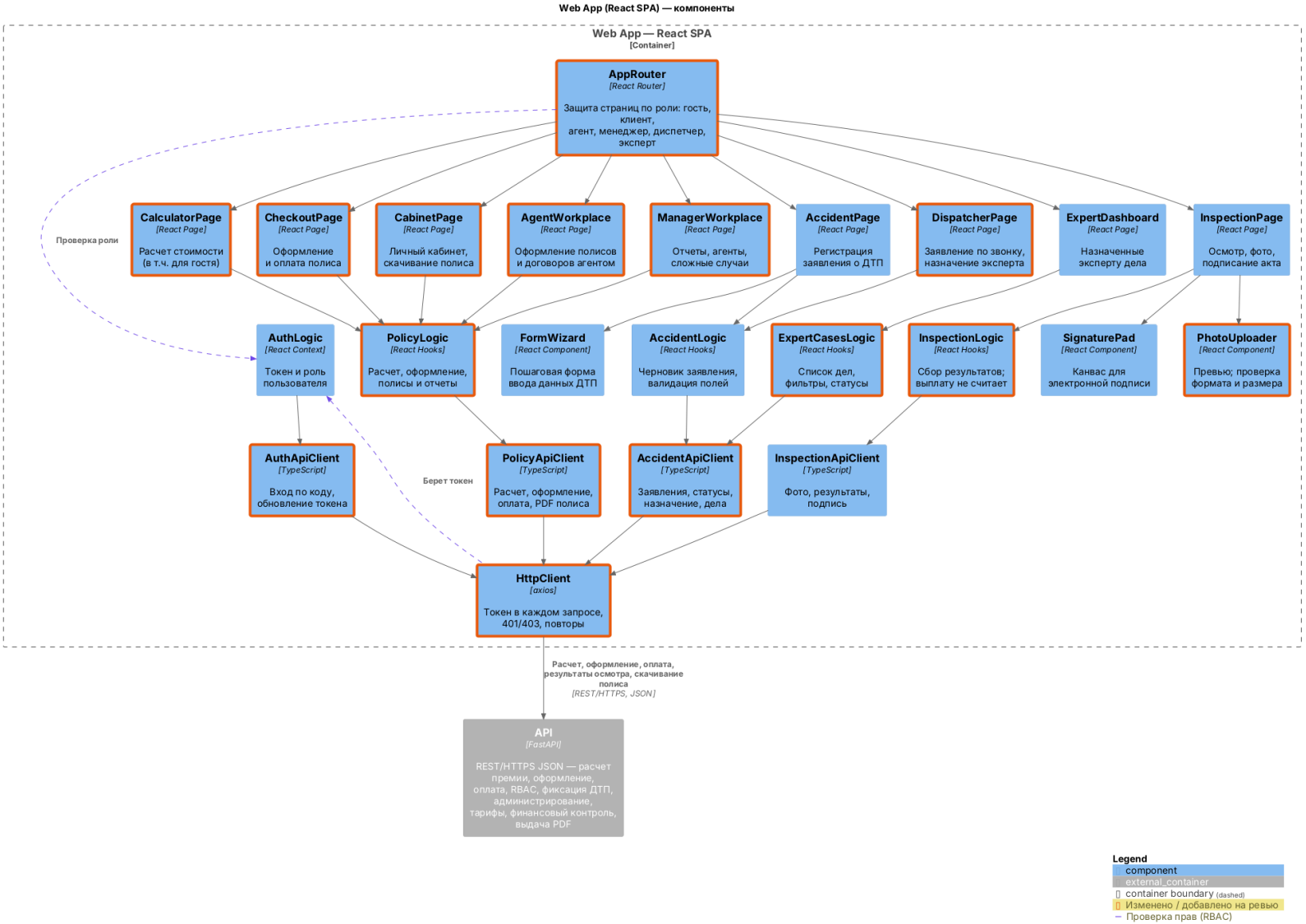


Рисунок 12 — Диаграмма компонентов Web App после ревью — вид в PlantUML

Описание компонентов

Web App — React SPA для клиентов и сотрудников фронт-линии. Единственная внешняя связь — вызовы API.

- AppRouter закрывает страницы по роли: гость, клиент, агент, менеджер, диспетчер, эксперт. Страницы: CalculatorPage, CheckoutPage, CabinetPage, AgentWorkplace, ManagerWorkplace, AccidentPage, DispatcherPage, ExpertDashboard, InspectionPage.
- **Логика и UI** — AuthLogic, PolicyLogic, AccidentLogic, ExpertCasesLogic, InspectionLogic, FormWizard, PhotoUploader, SignaturePad; API-клиенты — Auth, Policy, Accident, Inspection, все через HttpClient (axios) с токеном и обработкой 401/403.

Сводка изменений

№	Что было	Что стало и почему
1	Снаружи границы нарисован Web App, который вызывает свои же страницы	Контейнер не может вызывать сам себя — внешний Web App убран. Точка входа — AppRouter внутри контейнера; пользователей не рисуем.
2	Покрыт только контур ДТП	Добавлены CalculatorPage (в т.ч. для гостя), CheckoutPage (оформление и оплата), CabinetPage (кабинет, скачивание полиса), AgentWorkplace, ManagerWorkplace, а также PolicyLogic и PolicyApiClient.
3	Нет экрана диспетчера	Добавлен DispatcherPage: заявление по звонку и назначение эксперта через AccidentLogic и AccidentApiClient.
4	ExpertDashboard использует AccidentLogic	Добавлен ExpertCasesLogic: у эксперта свой сценарий — назначенные дела и фильтры.
5	По коду ветки (inspection_logic/strategies) InspectionLogic на фронте считает сумму выплаты: TotalLoss = рыночная стоимость × 0,85	Расчет выплаты с фронта убран: значение на клиенте можно подменить. На уровнях 1–2 расчета выплаты нет — место на бэкенде решаем отдельно (см. «Что уточнить»).
6	AuthLogic ходит в API напрямую, токен никуда не передается	Добавлены AuthApiClient и общий HttpClient (axios): токен в каждом запросе, обработка 401/403. Все API-клиенты работают через него.
7	Связь с API подписана «REST/HTTPS + JSON»	Подпись связи — дословно с уровня 2: «Расчет, оформление, оплата, результаты осмотра, скачивание полиса [REST/HTTPS, JSON]».
8	PhotoUploader только загружает фото	Добавлена проверка формата и размера до отправки.

Что поправить в коде

- inspection_logic/strategies: убрать расчет payout (Minor/Medium/TotalLoss) из фронта.
- Сделать общий HttpClient с интерсептором токена и перевести на него существующие API-клиенты.

Итог

После ревью у каждого контейнера одна диаграмма компонентов: API — Дима, Worker — Миша, Backoffice — Амелия, Web App — Никита. Все внешние связи перенесены с диаграммы контейнеров один в один (API — 12, Worker — 7, Backoffice — 1, Web App — 1; связи с пользователями на компонентных не рисуются), имена и подписи совпадают с уровнем 2. Диаграмму Димы намеренно не раздували: исходные компоненты сохранены, добавлено только необходимое для связей уровня 2 и проверки прав. Логика, которая была не на своем месте, перенесена: проверка полиса — на регистрацию заявления, очередь — в outbox DB_AUDIT без Redis, расчет выплаты убран с фронта.

Исправленные диаграммы (.png, .drawio и .puml) лежат в папке component_review рядом с отчетом и заменяют исходные в ветках.

Что уточнить в документах уровня 2 и командных отчетах

- Расчет выплаты по ДТП на уровнях 1–2 не описан — решить, в каком компоненте API он будет, и добавить в описание API.
- Брокер Celery: если оставляем Celery, записать на диаграмме контейнеров, что брокер — PostgreSQL (DB_AUDIT), а не Redis.
- Общий отчет, табл. 8: роли Web App и Backoffice расходятся с container.puml (администратор, бухгалтер, ИБ и поддержка — в Backoffice), базы названы BD_* вместо DB_*.
- Общий отчет, табл. 5–6: строки с Redis устарели — очередь теперь outbox в DB_AUDIT.